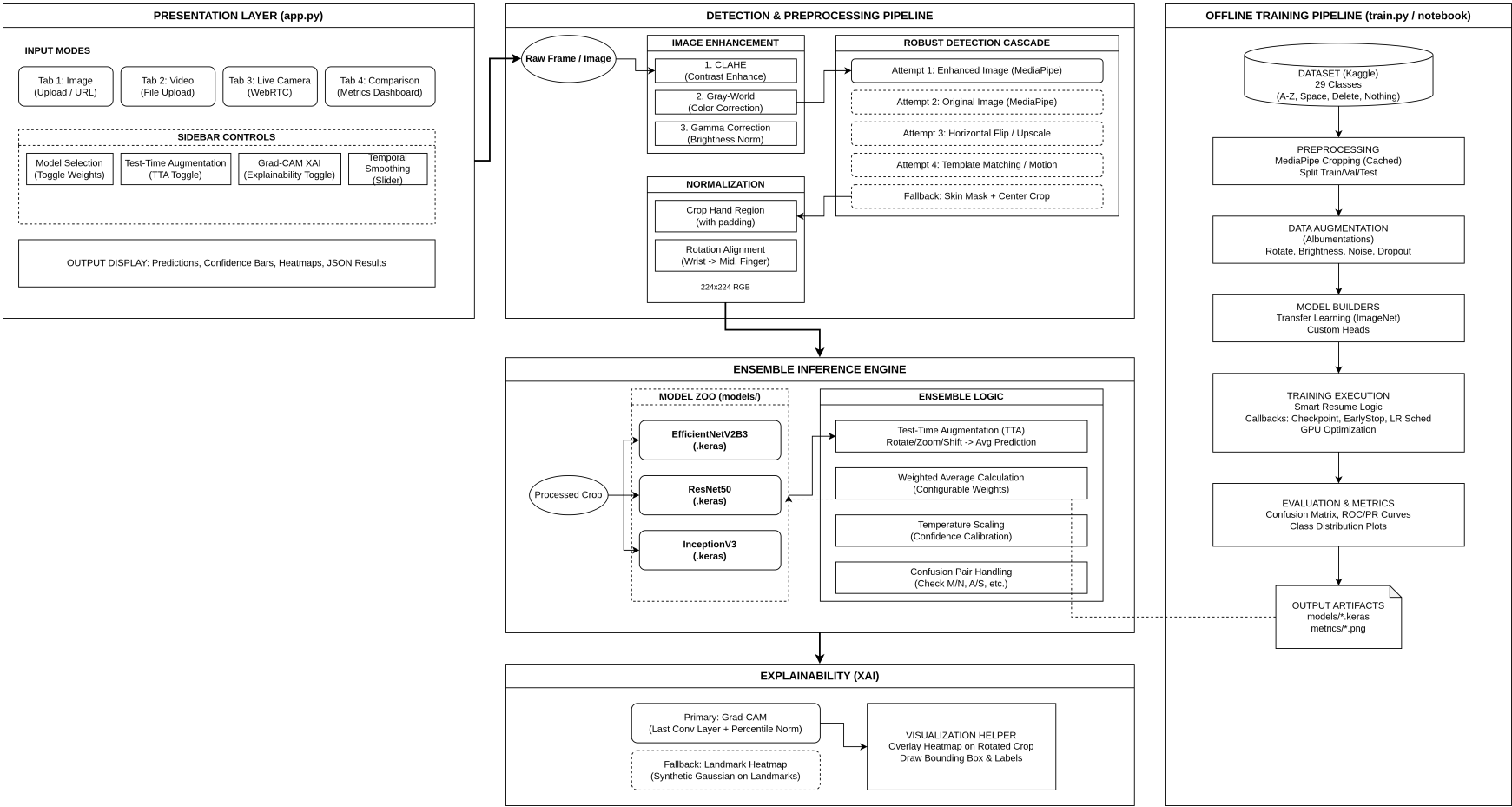




Component	Description
Streamlit App	Interactive web interface with multiple input modes
Model Ensemble	EfficientNetV2B3, ResNet50, and InceptionV3 (configurable)
Hand Detection	MediaPipe-based detection with robust fallback mechanisms
Training Pipeline	Complete training workflow with augmentation and evaluation

Major Enhancements in This Version

Category	Improvements
Image Preprocessing	CLAHE contrast enhancement, gray-world color correction, gamma correction
Hand Detection	Multiple detection variants, template/motion proposals, intelligent fallbacks
Rotation Handling	Automatic rotation normalization with multi-rotation fallback
Ensemble Inference	Weighted ensemble with TTA and temperature scaling for calibrated confidences
Explainability	Real Grad-CAM implementation with landmark-based XAI fallback

Key Features

Input Modes

- Image upload and URL fetching
- Video file processing with frame-by-frame analysis
- Live camera recognition via WebRTC
- Model comparison and analysis dashboard

Technical Capabilities

- Multi-model ensemble with configurable weights
- Test-Time Augmentation (TTA) for improved accuracy
- Temporal smoothing for video and live streams
- Confusion pair detection and handling
- Real-time performance optimization

About the Dataset

Dataset Overview

Attribute	Details
Total Classes	29
Letter Classes	26 (A-Z)
Special Classes	SPACE, DELETE, NOTHING
Data Organization	Separate folders per class
Test Set	Small set provided for real-world testing

Purpose and Context

The primary goal of this dataset is to bridge the communication gap between sign-language users and non-sign-language users. By providing tools for automatic ASL recognition, we aim to make communication more accessible and inclusive.

Acknowledgements

Thanks to all open dataset contributors and the sign language community for making this work possible.

Repository Structure

Main Files

File/Folder	Description
app.py	Streamlit application with all UI tabs and inference logic
class_names.json	Class labels for inference and training
requirements.txt	Python package dependencies
packages.txt	System-level package dependencies
train-asl-models-4.ipynb	Training notebook with experiments and visualizations
more code/	Additional preprocessing and helper scripts

Generated Outputs

Folder	Contents
metrics/	Visualizations, confusion matrices, performance dashboards
models/	Saved model files (.keras format) and checkpoints

Directory Tree

```
project-root/
|-- app.py
|-- class_names.json
|-- requirements.txt
|-- packages.txt
|-- train-asl-models-4.ipynb
|-- more code/
|   |-- preprocessing_helpers.py
|   |-- detection_utils.py
|-- metrics/
|   |-- class_distribution.png
|   |-- model_comparison.png
|   |-- confusion_matrices/
|-- models/
|   |-- efficientnet_v2b3.keras
|   |-- resnet50.keras
|   |-- inception_v3.keras
```

Installation

Prerequisites

Requirement	Details
Python Version	3.8 or higher
Operating System	macOS or Linux recommended for development
GPU Support	Optional but recommended for training

Installation Steps

1. Create a virtual environment (recommended):

```
python -m venv asl_env
source asl_env/bin/activate # On Windows: asl_env\Scripts\activate
```

2. Install dependencies:

```
python -m pip install -r requirements.txt
```

3. Install optional visualization packages:

```
pip install plotly kaleido
```

Important Notes

Note	Details
MediaPipe	Uses CPU (TFLite) for detection
TensorFlow	Uses GPU when available for training
Memory	Ensure sufficient RAM for model loading (minimum 8GB recommended)

Quick Start

Running the Application

1. Start the Streamlit server:

```
streamlit run app.py
```

2. Open your browser and navigate to the URL displayed (typically <http://localhost:8501>)

Application Tabs

Tab	Functionality
Image	Upload images or fetch from URL, run detection and ensemble inference
Video	Upload video files, process frames with optional temporal smoothing
Live Camera	Real-time recognition via WebRTC with ensemble or single-model mode
Model Comparison	View model details, training information, and confusion analysis

Sidebar Controls

Control	Options
Model Selection	Enable/disable individual models in the ensemble
TTA Toggle	Enable Test-Time Augmentation for improved accuracy
Grad-CAM Toggle	Enable visual explanations for predictions
Smoothing	Adjust temporal smoothing for video/live modes

Training Pipeline

Configuration

Training is controlled through a CONFIG dictionary with the following key parameters:

Parameter	Description
train_path	Path to training data directory
test_path	Path to test data directory
cropped_train_path	Path for MediaPipe-cropped images (cached)
models_path	Directory for saved models
metrics_path	Directory for evaluation outputs

Running Training

Option 1 - Using the training script:

```
python train.py
```

Option 2 - Using the Jupyter notebook:

```
jupyter notebook train-asl-models-4.ipynb
```

Training Features

Feature	Description
MediaPipe Cropping	Automatic hand cropping with caching
Augmentation	Albumentations-based data augmentation
Model Builders	ResNet50, InceptionV3, EfficientNetV2B3
Smart Resume	Automatic detection of resume vs. scratch training
Callbacks	Checkpointing, early stopping, LR scheduling
Visualization	3D class distributions, confusion matrices, ROC/PR curves

Resource Configuration

Setting	Purpose
CUDA_VISIBLE_DEVICES	Control GPU visibility
TF Thread Settings	Manage CPU thread allocation

Model Inference

Supported Model Formats

Format	Support Level
.keras	Full support
.h5	Full support
SavedModel	Inference-only (wrapped when possible)

Model Loading

Models are loaded from the models/ folder or downloaded from configured hub sources.

Configuration	Location
Class Labels	class_names.json
HuggingFace Token	Streamlit secrets (HF_TOKEN)
Ensemble Weights	Sidebar or session state

Inference Modes

Mode	Description
Single Model	Fast inference with one selected model
Ensemble	Weighted average across multiple models
Ensemble + TTA	Enhanced accuracy with test-time augmentation

Preprocessing and Detection

Image Preprocessing Pipeline

Step	Technique	Purpose
1	CLAHE	Contrast Limited Adaptive Histogram Equalization
2	Gray-World	Color correction for lighting normalization
3	Gamma Correction	Brightness adjustment toward target mean
4	Model-Specific	preprocess_input matching training conditions

Detection Robustness Strategy

Priority	Method	Condition
1	Enhanced Image	CLAHE + gray-world corrected
2	Original Image	Unmodified input
3	Flipped Image	Horizontal flip attempt

Priority	Method	Condition
4	Upscaled Image	1.5x resolution increase
5	Template Matching	For video sequences
6	Motion Proposals	Movement-based detection
7	Heuristic Fallback	Skin mask + centered crop

Rotation Normalization

Component	Details
Reference Points	Wrist and middle-finger MCP landmarks
Alignment Goal	Fingers pointing upward
Application Condition	Only when landmarks are reliable

Explainability and Visualizations

Grad-CAM Implementation

Feature	Description
Target Layer	Last convolutional layer
Normalization	Percentile-based for robust scaling
Fallback	Landmark-based synthetic heatmap when Grad-CAM is weak

Generated Visual Assets

File	Description
class_distribution.png	3D class distribution visualization
model_comparison.png	2D model performance comparison
model_comparison_3d.png	3D model performance comparison
*_confusion_matrix.png	Per-model confusion matrices
*_roc_curves_3d.png	3D ROC curve visualizations
*_pr_curves_3d.png	3D Precision-Recall curves

Detailed Code Overview

1. Streamlit Application (app.py)

Main Entry Point

Function	Purpose
main()	Sets up sidebar and tab navigation

Tab Functions

Function	Description
tab_image_upload()	Image upload, detection, rotation normalization, inference, Grad-CAM
tab_video_upload()	Video processing with temporal smoothing

Function	Description
tab_live_camera_webrtc()	WebRTC live processing via ASLVideoProcessor class
tab_model_comparison()	Model statistics and confusion analysis

Configuration Constants

Constant	Purpose
IMG_SIZE	Input image dimensions
PAD_FACTOR	Padding factor for hand crops
DEFAULT_ENSEMBLE_WEIGHTS	Initial model weights
DEFAULT_TEMPERATURE	Temperature scaling parameter

2. Hand Detection Module

Primary Detection Function

```
detect_hand_in_frame(frame_rgb, detector, enhance=True, allow_fallback=True)
```

Return Value	Description
crop_rgb	Cropped hand region
bbox	Bounding box coordinates
landmarks	MediaPipe hand landmarks
used_fallback	Boolean indicating fallback usage

Preprocessing Helpers

Function	Purpose
enhance_image()	CLAHE on LAB L-channel
grayworld_color_correction()	Gray-world channel scaling
gamma_correction()	Adaptive gamma toward target mean
normalize_hand_rotation()	Landmark-based rotation alignment

3. Model Inference Module

Core Functions

Function	Purpose
preprocess_for_inference()	Model-specific preprocessing
predict_with_model()	Thread-safe single model inference
single_model_predict()	Optimized real-time prediction
ensemble_predict()	Weighted average across models
ensemble_predict_tta()	Ensemble with test-time augmentation

Confusion Handling

Component	Description
CONFUSION_PAIRS	Dictionary of commonly confused letter pairs
handle_confusion_pairs()	Detection and annotation of confusions

4. Explainability Module

Core Functions

Function	Purpose
make_gradcam_heatmap()	Compute Grad-CAM for top class
create_landmark_based_heatmap()	Synthetic heatmap fallback

Visualization Helpers

Function	Purpose
draw_hand_overlay()	Place heatmap on crop
draw_full_rotated_overlay()	Place heatmap on full image
draw_hand_overlay_on_rotated_crop()	Combined overlay with annotations

5. Video Processing

ASLVideoProcessor Class

Method	Purpose
recv()	Process incoming video frames
_preprocess_for_model()	Per-model preprocessing
_single_predict()	Single model inference
_ensemble_predict()	Ensemble inference

Feature	Description
Frame Skipping	Configurable for performance
Temporal Smoothing	Reduces prediction jitter
State Management	Maintains detection and model cache

6. Training Pipeline

Key Components

Component	Description
HandCropper	MediaPipe-based hand extraction
process_dataset_with_mediapipe()	Batch processing with caching
AugmentedDataGenerator	Albumentations wrapper

Model Builders

Function	Architecture
build_resnet50()	ResNet50 with custom head
build_inception()	InceptionV3 with custom head
build_efficientnet()	EfficientNetV2B3 with custom head

Training and Evaluation

Function	Purpose
train_model()	Complete training orchestration

Function	Purpose
evaluate_model()	Per-class metrics and visualizations
test_models_before_training()	Baseline performance check

7. Utility Functions

Model Loading

```
load_model_any_format(path)
```

Format Attempted	Fallback Strategy
.keras	Direct load
.h5	Keras load
SavedModel	Signature wrapping or TFSMLayer

Logging

Class	Purpose
Logger	Text and visual artifact logging to metrics/

Project Todo List

1. Data Collection and Preparation

- ✓ Download ASL dataset from Kaggle
- ✓ Verify dataset integrity
- ✓ Split dataset into train/val/test
- ✓ Run MediaPipe cropping on dataset
- ✓ Cache cropped images
- ✓ Analyze class distribution
- ✓ Create data augmentation pipeline

2. Model Development

- ✓ Implement EfficientNetV2B3 builder
- ✓ Implement ResNet50 builder
- ✓ Implement InceptionV3 builder
- ✓ Configure model hyperparameters
- ✓ Set up training callbacks
- ✓ Implement ensemble logic
- ✓ Add TTA support
- ✓ Implement temperature scaling

3. Training Execution

- ✓ Train EfficientNetV2B3 model
- ✓ Train ResNet50 model
- ✓ Train InceptionV3 model
- ✓ Monitor training progress
- ✓ Save best model checkpoints
- ✓ Fine-tune models if needed
- ✓ Export final models to .keras

4. Evaluation and Metrics

- ✓ Generate confusion matrices
 - ✓ Calculate accuracy metrics
 - ✓ Create ROC curves
 - ✓ Create PR curves
 - ✓ Generate class distribution plots
 - ✓ Create model comparison charts
 - ✓ Identify confusion pairs
 - ✓ Document evaluation results
-

5. Application Development (app.py)

- ✓ Implement main() entry point
 - ✓ Create sidebar controls
 - ✓ Implement Image upload tab
 - ✓ Implement Video upload tab
 - ✓ Implement Live Camera tab
 - ✓ Implement Model Comparison tab
 - ✓ Add Grad-CAM visualization
 - ✓ Add temporal smoothing
 - ✓ Implement error handling
 - ✓ Optimize for performance
-

6. Preprocessing Pipeline

- ✓ Implement CLAHE enhancement
 - ✓ Implement gray-world correction
 - ✓ Implement gamma correction
 - ✓ Create detection fallback chain
 - ✓ Implement rotation normalization
 - ✓ Add template matching fallback
 - ✓ Add motion-based proposals
-

7. Explainability (XAI)

- ✓ Implement Grad-CAM
 - ✓ Add percentile normalization
 - ✓ Create landmark-based fallback
 - ✓ Implement overlay drawing
 - ✓ Add bounding box visualization
 - ✓ Add landmark visualization
-

8. Documentation

- ✓ Write README.md
 - ✓ Document installation steps
 - ✓ Create usage examples
 - ✓ Document API functions
 - ✓ Add inline code comments
 - ✓ Create architecture diagrams
 - ✓ Write troubleshooting guide
-

9. Testing and Quality

- ✓ Test image upload functionality
- ✓ Test video processing
- ✓ Test live camera mode
- ✓ Test model loading
- ✓ Test ensemble predictions
- ✓ Test edge cases
- ✓ Performance testing
- ✓ Cross-browser testing

10. Deployment

- ✓ Prepare requirements.txt
- ✓ Prepare packages.txt
- ✓ Test local deployment
- ✓ Configure Streamlit secrets
- ✓ Deploy to Streamlit Cloud
- ✓ Set up model hosting
- ✓ Monitor deployed app
- ✓ Gather user feedback

Progress Summary

Category	Total Tasks	Completed	In Progress	Not Started
Data Collection	7	7	0	0
Model Development	8	8	0	0
Training Execution	7	7	0	0
Evaluation	8	8	0	0
Application Development	10	10	0	0
Preprocessing	7	7	0	0
Explainability	6	6	0	0
Documentation	7	7	0	0
Testing	8	8	0	0
Deployment	8	8	0	0
Future Enhancements	7	0	0	7
TOTAL	83	76	0	7